# Reinforcement Learning

## Multi-agent Systems & Agentic AI

Lecture Notes

Easter Term 2026

Marta Grzeskiewicz
mg2192@cam.ac.uk

# Course Map

## Lectures

1. **Foundations of RL**
   MDP, policies, value functions
2. **Policy Gradient Methods**
   REINFORCE, baselines, advantage
3. **GRPO**
   Group Relative Policy Optimization
4. **Theory**
   Variance, bias, convergence
5. **Agentic Alignment**
   RLHF as system, safety, failures

## Running Thread

Each lecture builds toward a single question:

*How do we train an LLM to behave as intended?*

**Lectures 1–2**: classical tools
**Lecture 3**: GRPO + RLHF bridge
**Lecture 4**: why it (sometimes) works
**Lecture 5**: alignment as a system

# Lecture 1

Foundations of Reinforcement Learning

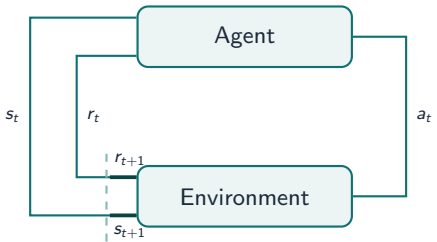
# Why Reinforcement Learning?

**Supervised learning** needs labelled targets.

Many desirable behaviours are **easier to evaluate than to demonstrate**:

- Winning a chess position
- Writing a helpful answer
- Following a user's instruction safely

**RL**: learn from **scalar feedback** (reward) obtained by acting.



# Example: AlphaGo

# The Markov Decision Process (MDP)

**Definition**

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

# The Markov Decision Process (MDP)

**Definition**

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

| Symbol | Meaning |
|---|---|
| $\mathcal{S}$ | state space |
| $\mathcal{A}$ | action space |
| $P(s' \mid s, a)$ | transition kernel |
| $R(s, a)$ | expected reward |
| $\gamma \in [0, 1)$ | discount factor |

# The Markov Decision Process (MDP)

**Definition**

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

| Symbol | Meaning |
|---|---|
| $\mathcal{S}$ | state space |
| $\mathcal{A}$ | action space |
| $P(s' \mid s, a)$ | transition kernel |
| $R(s, a)$ | expected reward |
| $\gamma \in [0, 1)$ | discount factor |

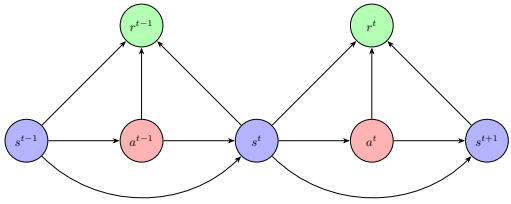



**Figure 1:** MDP Transition Loop

# The Markov Decision Process (MDP)

## Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

| Symbol | Meaning |
|---|---|
| $\mathcal{S}$ | state space |
| $\mathcal{A}$ | action space |
| $P(s' \mid s, a)$ | transition kernel |
| $R(s, a)$ | expected reward |
| $\gamma \in [0, 1)$ | discount factor |

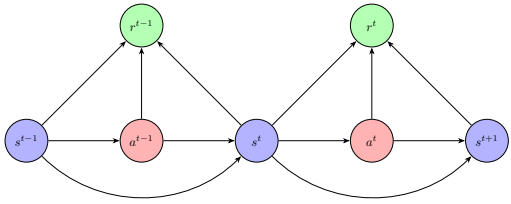



**Figure 1:** MDP Transition Loop

## Markov property

$P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$

The future is conditionally independent of the past given the present state.

# Policies

**Definition**

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

# Policies

**Definition**

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

**Stochastic policy**

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

# Policies

**Definition**

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

**Stochastic policy**

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

**Deterministic policy**

$$\mu : \mathcal{S} \to \mathcal{A}$$

- Special case: $\pi(a|s) = \mathbf{1}[a = \mu(s)]$
- Useful for continuous control
- Not directly applicable to LLMs

# Policies

**Definition**

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

**Stochastic policy**

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

**Deterministic policy**

$$\mu : \mathcal{S} \to \mathcal{A}$$

- Special case: $\pi(a|s) = \mathbf{1}[a = \mu(s)]$
- Useful for continuous control
- Not directly applicable to LLMs

Later in this course: $\pi_\theta$ is a language model parameterised by weights $\theta$.

# The RL Objective

Define the **return** (total discounted reward) from timestep $t$:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

# The RL Objective

Define the **return** (total discounted reward) from timestep $t$:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **RL objective** is to maximise:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{T} \gamma^t r_t \right]$$

# The RL Objective

Define the **return** (total discounted reward) from timestep $t$:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **RL objective** is to maximise:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{T} \gamma^t r_t \right]$$

- $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ is a **trajectory**
- $\gamma < 1$ ensures convergence for infinite horizons
- Choice of $\gamma$ encodes *time preference*

# Trajectories and Their Distribution

**Trajectory distribution**

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t)\, \pi_\theta(a_t \mid s_t)$$

- $\rho_0$: initial state distribution
- The policy influences which trajectories are visited
- **Key insight**: gradient of $J(\theta)$ must account for the changing distribution

# Value Functions

**State-value function**

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state $s$ following $\pi$.

# Value Functions

## State-value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state $s$ following $\pi$.

## Action-value function

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Expected return taking action $a$ from $s$, then following $\pi$.

# Value Functions

## State-value function

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Expected return starting from state $s$ following $\pi$.

## Action-value function

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Expected return taking action $a$ from $s$, then following $\pi$.

## Advantage function

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

How much better is action $a$ than the *average* action in state $s$?

# State-Value Functions

**State-value** functions $V^{\pi}(s)$ give the 'value' of state $s$ when following the policy $\pi$:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t | S_t = s\right]$$

# State-Value Functions

**State-value** functions $V^{\pi}(s)$ give the 'value' of state $s$ when following the policy $\pi$:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t | S_t = s\right]$$

The return ($u$) can be recursively defined as:

$$\begin{aligned} G_t &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + ...) \\ &= r_t + \gamma G_{t+1} \end{aligned}$$

# State-Value Functions

**State-value** functions $V^{\pi}(s)$ give the 'value' of state $s$ when following the policy $\pi$:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t | S_t = s\right]$$

The return ($u$) can be recursively defined as:

$$\begin{aligned} G_t &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + ...) \\ &= r_t + \gamma G_{t+1} \end{aligned}$$

Therefore:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[r_t + \gamma G_{t+1} | S_t = s\right]$$

# The Bellman Equation

$$V^{\pi}(s) = \mathbb{E}_{\pi}[r_t + \gamma G_{t+1} \mid S_t = s] \tag{1}$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s_t) \sum_{s' \in \mathcal{S}} P(s'|s,a) \left[R(s,a) + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s']\right] \tag{2}$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a) + \gamma V^{\pi}(s')\right] \tag{3}$$

The last equation is known as the **Bellman equation** in honour of Richard Bellman.

# State-Action Value Function

**State-action** value functions $Q^{\pi}(s, a)$ are an extension on the *State* value functions. They condition the expected return on the current state and the action taken:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[G_t | S_t = s, A_t = a\right]$$

The *state-action* value Bellman equation is therefore:

$$Q^{\pi}(s, a) = \sum_{s' \in S} P(s'|s, a)\left[\mathcal{R}(s, a) + \gamma V^{\pi}(s')\right]$$

# Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

# Why the Advantage Function Matters

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- $A^{\pi}(s,a) > 0 \Rightarrow$ action $a$ is **better than average**; increase its probability

# Why the Advantage Function Matters

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- $A^{\pi}(s,a) > 0 \Rightarrow$ action $a$ is **better than average**; increase its probability
- $A^{\pi}(s,a) < 0 \Rightarrow$ action $a$ is **worse than average**; decrease its probability

# Why the Advantage Function Matters

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

- $A^\pi(s,a) > 0 \Rightarrow$ action $a$ is **better than average**; increase its probability
- $A^\pi(s,a) < 0 \Rightarrow$ action $a$ is **worse than average**; decrease its probability
- $A^\pi(s,a) = 0 \Rightarrow$ indifferent; no update needed

# Why the Advantage Function Matters

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

- $A^\pi(s,a) > 0 \Rightarrow$ action $a$ is **better than average**; increase its probability
- $A^\pi(s,a) < 0 \Rightarrow$ action $a$ is **worse than average**; decrease its probability
- $A^\pi(s,a) = 0 \Rightarrow$ indifferent; no update needed

**Baseline intuition**

Subtracting $V^\pi(s)$ from $Q^\pi(s,a)$ gives a *centered* signal.
This is the single most important variance-reduction idea in RL.

# RL as Optimisation over Distributions

**Key Reframing**

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \int p_{\pi_\theta}(\tau)\left(\sum_{t=0}^{T} \gamma^t r_t\right) d\tau$$

- $\theta$ shapes the *distribution* $p_\theta(\tau)$; we optimise over distributions
- This is the fundamental challenge: the expectation depends on $\theta$
- Standard autodiff cannot directly differentiate through *sampling*
- Solution: the **log-derivative trick** (Lecture 2)

# Why Not Value Iteration?

Classical **value-based methods** (Q-learning, value iteration):

- Enumerate $\mathcal{S} \times \mathcal{A}$ — infeasible for LLMs ($|\mathcal{A}| \approx 50{,}000$ tokens, sequences of length $\sim 10^3$)
- Greedy extraction breaks differentiability
- Poor generalisation across states

**LLM Scale**

A single output sequence is an action. The action space is combinatorially huge.

Direct policy optimisation with gradient descent is the *only* scalable approach.

# Lecture 1 — Summary

## Core take-aways

1. RL formalises **sequential decision-making** via MDPs
2. A **policy** $\pi(a|s)$ is what we learn; the **objective** is to maximise the expected discounted return:
$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{T} \gamma^t r_t \right]$$
3. **Value functions** $V^\pi$, $Q^\pi$, $A^\pi$ measure long-run quality
4. The **advantage function** is the key quantity for policy updates
5. RL = optimise a policy over a *distribution of trajectories*

Next: how do we compute $\nabla_\theta J(\theta)$?

# Lecture 2

Policy Gradient Methods

# The Policy Gradient Objective

We want $\nabla_\theta J(\theta)$ where:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{T} \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

**Challenge**: $p_\theta(\tau)$ depends on $\theta$, so we cannot simply push $\nabla_\theta$ inside the integral naively.

# The Policy Gradient Objective

We want $\nabla_\theta J(\theta)$ where:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{T} \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

**Challenge**: $p_\theta(\tau)$ depends on $\theta$, so we cannot simply push $\nabla_\theta$ inside the integral naively.

**Log-derivative (REINFORCE) trick**

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$$

because $\nabla_\theta \log p = \frac{\nabla_\theta p}{p}$.

# The Log-Derivative Trick

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int \nabla_\theta p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot G(\tau)]
\end{aligned}
$$

# The Log-Derivative Trick

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int \nabla_\theta p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot G(\tau)]
\end{aligned}
$$

Since $\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_t \log P(s_{t+1}|s_t, a_t) + \sum_t \log \pi_\theta(a_t|s_t)$, and only the last term depends on $\theta$:

$$
\boxed{\nabla_\theta J(\theta) \propto \mathbb{E}_{\tau \sim p_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G(\tau)\right]}
$$

# The REINFORCE Algorithm

**REINFORCE (Williams, 1992)**

1. Sample trajectory $\tau \sim \pi_\theta$
2. Compute return $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ for each step $t$
3. Update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

# The REINFORCE Algorithm

**REINFORCE (Williams, 1992)**

1. Sample trajectory $\tau \sim \pi_\theta$
2. Compute return $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ for each step $t$
3. Update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

- Unbiased gradient estimate
- No model of $P$, no value function required

# Exploration vs. Exploitation

### The core tension

**Exploit**: repeat actions already known to give high reward.

**Explore**: try new actions that might be better — but carry risk.

### LLM connection

Temperature $T$ controls this trade-off at inference time.

High $T \rightarrow$ flat $\pi_\theta \rightarrow$ exploration.
Low $T \rightarrow$ sharp $\pi_\theta \rightarrow$ exploitation.

### How REINFORCE handles it implicitly

- A *stochastic* policy $\pi_\theta(a|s)$ explores by construction — it never deterministically commits to one action.
- As training progresses, high-reward actions accumulate probability mass: the policy becomes increasingly deterministic.
- Left unchecked, this causes **premature convergence**: the agent stops exploring before reaching the global optimum.

# Code: REINFORCE — Policy and Episode

```python
import torch, torch.nn as nn

class Policy(nn.Module):
    def __init__(self, n_obs, n_act):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_obs, 64),
            nn.ReLU(),
            nn.Linear(64, n_act),
        )

    def forward(self, obs):
        logits = self.net(obs)
        # stochastic policy = Categorical
        return torch.distributions.Categorical(
            logits=logits
        )

def collect_episode(env, policy):
    obs = env.reset()
    log_probs, rewards = [], []
    done = False
    while not done:
        dist = policy(torch.tensor(obs, dtype=torch.float))
        action = dist.sample()
        log_probs.append(dist.log_prob(action))
        obs, r, done, _ = env.step(action.item())
        rewards.append(r)
    return log_probs, rewards
```

**Key lines:**

- `Categorical(logits=)` is $\pi_\theta(a|s)$
- `dist.sample()` — stochastic action
- `dist.log_prob(a)` — the score function $\nabla_\theta \log \pi$ will come from autodiff on this
- No value network, no critic

# Code: REINFORCE — Update Step

```python
def reinforce_update(log_probs, rewards,
                     optimizer, gamma=0.99):
    # 1. Discounted returns
    G, returns = 0.0, []
    for r in reversed(rewards):
        G = r + gamma * G
        returns.insert(0, G)
    returns = torch.tensor(returns, dtype=torch.float)

    # 2. Standardise (baseline = mean return)
    returns = (returns - returns.mean()) \
              / (returns.std() + 1e-8)

    # 3. Policy gradient loss
    log_probs_t = torch.stack(log_probs)
    loss = -(log_probs_t * returns).sum()

    # 4. SGD step
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()
```

**What to watch when you run this:**

- `loss` fluctuates wildly — this *is* the variance problem
- Gradient norms vary by $10\times$ across episodes
- Convergence is slow: hundreds of episodes for CartPole

**Reference**

Spinning Up:
`spinningup.openai.com`
Clean minimal PyTorch implementations of REINFORCE $\rightarrow$ PPO.

# The Variance Problem

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}$$

**Sources of variance:**

- Returns $G_t$ accumulate noise over the whole trajectory
- Stochastic transitions multiply uncertainty
- Long horizons: variance scales as $O(T^2)$

**Consequence**

In practice REINFORCE needs thousands of samples and struggles.

# Baselines for Variance Reduction

**Baseline theorem**

For any function $b(s)$ that does **not** depend on $a$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \big(G_t - b(s_t)\big)\right]$$

The estimate remains **unbiased**.

**Why?** $\mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)] = 0$ because:

$$\sum_a \pi_\theta(a|s)\, \nabla_\theta \log \pi_\theta(a|s) = \nabla_\theta \sum_a \pi_\theta(a|s) = \nabla_\theta 1 = 0$$

# Proof: Baseline has Zero Expectation

$$\begin{aligned}
\mathbb{E}_\pi\Big[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)\Big] &= \int \pi_\theta(a|s)\, \nabla_\theta \log \pi_\theta(a|s) \cdot b(s)\; da \\
&= \int \nabla_\theta \pi_\theta(a|s) \cdot b(s)\; da \\
&= b(s)\, \nabla_\theta \underbrace{\int \pi_\theta(a|s)\; da}_{=1} = 0
\end{aligned}$$

**A baseline subtracts a constant from the reward signal without changing the gradient direction.**

Common choice: $b(s_t) = V^\pi(s_t)$, giving the **advantage** $G_t - V^\pi(s_t) \approx A^\pi(s_t, a_t)$.

# Policy Gradient with Advantage

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A^\pi(s_t, a_t)\right]$$

## Advantage Actor-Critic (A2C)

- **Actor**: policy $\pi_\theta$ — what to do
- **Critic**: value estimator $\hat{V}_\phi(s)$ — how good is the current state
- Advantage: $\hat{A}(s_t, a_t) = r_t + \gamma \hat{V}_\phi(s_{t+1}) - \hat{V}_\phi(s_t)$

## Problem in LLMs

Learning a reliable critic $\hat{V}_\phi$ for natural-language states is *hard*.
GRPO (Lecture 3) eliminates the critic entirely.

# Monte Carlo vs. Bootstrapping

**Monte Carlo**

$G_t = \sum_{k \geq t} \gamma^{k-t} r_k$

- Unbiased
- High variance
- Requires complete episodes

**Temporal Difference (TD)**

$G_t \approx r_t + \gamma \hat{V}(s_{t+1})$

- Biased (uses estimate)
- Lower variance
- Works online

**GAE** (Generalised Advantage Estimation) interpolates via $\lambda \in [0, 1]$:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

# Credit Assignment Noise

**Problem**

Which actions caused the high return? In a long trajectory, early and late rewards are entangled.

**Causal structure**: reward at $t$ should only influence updates for actions $a_{t' \leq t}$:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \underbrace{\sum_{k \geq t} \gamma^{k-t} r_k}_{G_t}\right]$$

REINFORCE already captures this via $G_t$ (future rewards only); the baseline sharpens it further.

**Why this rules out a replay buffer**

A replay buffer stores transitions $(s_t, a_t, r_t, s_{t+1})$ collected under an *old* policy $\pi_{\theta_{\text{old}}}$.

When we replay them under $\pi_\theta \neq \pi_{\theta_{\text{old}}}$, the credit assignment is doubly broken:

- $G_t$ was computed under $\pi_{\theta_{\text{old}}}$ — the return reflects the old policy's downstream behaviour, not what $\pi_\theta$ would have done after $a_t$.
- The score $\nabla_\theta \log \pi_\theta(a_t|s_t)$ now pushes $\theta$ based on a reward signal that was never causally linked to $\pi_\theta$.

**Upshot**: policy gradient must be on-policy, or use importance weighting — the cost GRPO avoids by treating each prompt as a single-step reward.

# Towards Stable Policy Updates: PPO

**The problem with large gradient steps**

A big update to $\theta$ can move $\pi_\theta$ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

**Policy collapse**: a few bad steps can destroy a well-trained policy irreversibly.

# Towards Stable Policy Updates: PPO

**The problem with large gradient steps**

A big update to $\theta$ can move $\pi_\theta$ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

**Policy collapse**: a few bad steps can destroy a well-trained policy irreversibly.

**TRPO** (2015) formalises this as a KL-constrained optimisation problem.

# Towards Stable Policy Updates: PPO

**The problem with large gradient steps**

A big update to $\theta$ can move $\pi_\theta$ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

**Policy collapse**: a few bad steps can destroy a well-trained policy irreversibly.

**TRPO** (2015) formalises this as a KL-constrained optimisation problem.

**Proximal Policy Optimisation (PPO) (Schulman et al., 2017)**

Replace the constraint with a **clipped surrogate objective**:

$$J^{\mathrm{PPO}}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t A_t,\ \mathrm{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\, A_t\right)\right]$$

where $\rho_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ is the **importance ratio** — how much the policy has moved.

The clip removes the incentive to push $\rho_t$ outside $[1-\varepsilon,\ 1+\varepsilon]$, bounding each step.

# Lecture 2 — Summary

**Core take-aways**

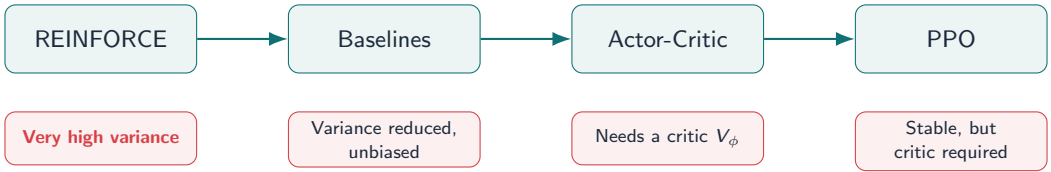
1. **Log-derivative trick**: $\nabla_\theta J = \mathbb{E}\big[\nabla_\theta \log \pi_\theta \cdot G\big]$
2. **REINFORCE**: unbiased, but high-variance gradient estimator
3. **Baselines** reduce variance without introducing bias
4. **Advantage** $A^\pi(s, a)$ is the centred signal of choice
5. **Actor-Critic**: use a learned $V^\pi$ as baseline — but critic training is hard

Next: eliminate the critic entirely using group comparisons.

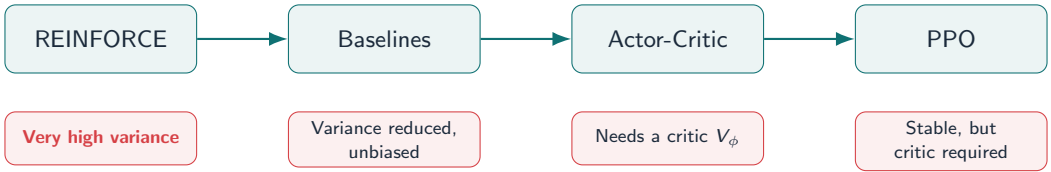
# Lecture 3

Group Relative Policy Optimization (GRPO)

# Recap


REINFORCE
Baselines
Actor-Critic
PPO
Very high variance
Variance reduced, unbiased
Needs a critic $V_\phi$
Stable, but critic required

# Recap




**The unresolved question**

PPO works — but it requires a **critic network** $V_\phi$ (same size as the policy) trained via TD.

Today: can we do better?

# PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

# PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

### 1. Critic update (TD)

Train $V_\phi$ to predict $V^\pi(s)$ by minimising the **TD error**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Bootstraps: the target uses the current $V_\phi$ estimate of the next state.

# PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

## 1. Critic update (TD)

Train $V_\phi$ to predict $V^\pi(s)$ by minimising the **TD error**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Bootstraps: the target uses the current $V_\phi$ estimate of the next state.

## 2. Actor update (policy gradient)

Update $\pi_\theta$ using the clipped surrogate:

$$J^{\text{PPO}} = \mathbb{E}_t\Big[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big]$$

$\rho_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the importance ratio and $\hat{A}_t$ comes from the critic.

# PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

### 1. Critic update (TD)

Train $V_\phi$ to predict $V^\pi(s)$ by minimising the **TD error**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Bootstraps: the target uses the current $V_\phi$ estimate of the next state.

### 2. Actor update (policy gradient)

Update $\pi_\theta$ using the clipped surrogate:

$$J^{\text{PPO}} = \mathbb{E}_t\Big[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big]$$

$\rho_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the importance ratio and $\hat{A}_t$ comes from the critic.

**TD** solves: *how good is this state?* **Policy gradient** solves: *which direction should $\theta$ move?*

# Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid

# Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

# Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

$\lambda = 0$

**One-step TD**

$\hat{A}_t = \delta_t$

**Low variance**

**High bias**

Trusts the critic immediately after one step.

# Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

$\lambda = 0$

**One-step TD**

$\hat{A}_t = \delta_t$

**Low variance**

**High bias**

Trusts the critic immediately after one step.

$\lambda = 1$

**Monte Carlo**

$\hat{A}_t = G_t - V_\phi(s_t)$

**Unbiased**

**High variance**

Uses all real rewards; never relies on the critic for future steps.

# Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

$\lambda = 0$

**One-step TD**

$\hat{A}_t = \delta_t$

**Low variance**

**High bias**

Trusts the critic immediately after one step.

$\lambda = 1$

**Monte Carlo**

$\hat{A}_t = G_t - V_\phi(s_t)$

**Unbiased**

**High variance**

Uses all real rewards; never relies on the critic for future steps.

$\lambda \approx 0.95$

**GAE (PPO default)**

$\hat{A}_t = \sum_k (\gamma\lambda)^k \delta_{t+k}$

**Low bias**

**Moderate variance**

Observes ≈20 real steps before bootstrapping. Critic errors get diluted.

$$\hat{A}_t = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}, \qquad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

**The core problem**

The critic adds a **full copy** of the model in memory, doubles the training complexity, and is the most common source of instability in PPO.

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

**The core problem**

The critic adds a **full copy** of the model in memory, doubles the training complexity, and is the most common source of instability in PPO.

**The key question**

Can we estimate a useful baseline **without training $V_\phi$ at all**?

# The Critic is the Bottleneck

**Why critic training is hard for LLMs:**

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

**The core problem**

The critic adds a **full copy** of the model in memory, doubles the training complexity, and is the most common source of instability in PPO.

**The key question**

Can we estimate a useful baseline **without training $V_\phi$ at all**?

**The answer**

Yes – replace the critic with the empirical group mean. This is GRPO (Shao et al 2024).

# PPO vs. GRPO: What Changes

| | PPO | GRPO |
|---|---|---|
| *Models loaded in memory:* | | |
| Policy $\pi_\theta$ (being trained) | ✓ | ✓ |
| Reference policy $\pi_{\text{old}}$ (frozen SFT) | ✓ | ✓ |
| Critic $V_\phi$ (value network, same size as policy) | ✓ | × |
| Reward model $r_\phi$ | ✓ | ✓ / ×† |
| **Total models in memory** | **4** | **3** (or **2**†) |
| Critic / value function training required | **Yes** | **No** |
| Advantage estimation | TD error via $V_\phi$ | Group mean $\bar{r}$ |
| Approx. GPU memory (70B, bf16) | ∼1.6 TB | ∼1.2 TB |

† When rewards are *verifiable* (maths, code), the reward model is replaced by a rule-based function — no separate model needed. DeepSeek-R1 uses this approach.

# Real-World Deployments

GRPO is now the standard RL algorithm for frontier reasoning models:

## DeepSeekMath (2024)

**Task**: mathematical reasoning
**Result**: state-of-the-art on MATH benchmark using GRPO, outperforming PPO-trained models of the same size

**Key finding**: group-relative advantages work especially well when rewards are *verifiable* (correct/incorrect) rather than learned

Shao et al., arXiv:2402.03300

## DeepSeek-R1 (2025)

**Task**: general reasoning (math, code, science)
**Result**: competitive with OpenAI o1 at a fraction of the training cost

**Architecture**: GRPO at every RL stage; no critic; rule-based verifiable rewards — **only 2 models in memory**

DeepSeek-AI, arXiv:2501.12948

# Broader Adoption

**Models trained with GRPO or direct variants:**

- **Kimi k1.5** (Moonshot AI, 2025): long-context RL with group-relative rewards; arXiv:2501.12599
- **Qwen2.5-Math** (Alibaba, 2024): GRPO for mathematical chain-of-thought reasoning; arXiv:2409.12122
- **Open-source reproductions**: Light-R1, SimpleRL, and others confirming results with $<8$ GPU training runs

**Paradigm shift**

Before GRPO: RL for LLMs required large research infrastructure.

After GRPO: a single research group can run competitive RL training on a cluster of 8 GPUs.

**GRPO democratised LLM RL.**

# Key Idea: Relative Comparisons

**Insight**

We don't need the *absolute* quality of a response.

We need to know: **is this response better or worse than typical?**

# Key Idea: Relative Comparisons

**Insight**

We don't need the *absolute* quality of a response.

We need to know: **is this response better or worse than typical?**

**Human analogy**: a teacher grades an essay not by a fixed scale, but relative to other essays submitted. **GRPO**: for each prompt $q$, sample $K$ responses and rank them *against each other*.

# Key Idea: Relative Comparisons

**Insight**

We don't need the *absolute* quality of a response.

We need to know: **is this response better or worse than typical?**

**Human analogy**: a teacher grades an essay not by a fixed scale, but relative to other essays submitted. **GRPO**: for each prompt $q$, sample $K$ responses and rank them *against each other*.

- No separate critic required
- Baseline is computed from the group itself
- Stable across training

# Step 1: Sample a Group of Responses

Given prompt $q$ (a context / instruction):

$$a_1, a_2, \ldots, a_K \overset{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- $K$ is the **group size** (e.g. $K = 8$ or $K = 16$)

# Step 1: Sample a Group of Responses

Given prompt $q$ (a context / instruction):

$$a_1, a_2, \ldots, a_K \overset{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- $K$ is the **group size** (e.g. $K = 8$ or $K = 16$)
- Responses are sampled from the **old policy** $\pi_{\text{old}}$ (policy before current update)

# Step 1: Sample a Group of Responses

Given prompt $q$ (a context / instruction):

$$a_1, a_2, \ldots, a_K \overset{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- $K$ is the **group size** (e.g. $K = 8$ or $K = 16$)
- Responses are sampled from the **old policy** $\pi_{\text{old}}$ (policy before current update)
- Each $a_i$ receives a reward $r_i = R(q, a_i)$ from a reward model

# Step 1: Sample a Group of Responses

Given prompt $q$ (a context / instruction):

$$a_1, a_2, \ldots, a_K \overset{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- $K$ is the **group size** (e.g. $K = 8$ or $K = 16$)
- Responses are sampled from the **old policy** $\pi_{\text{old}}$ (policy before current update)
- Each $a_i$ receives a reward $r_i = R(q, a_i)$ from a reward model

**In LLM practice**

$q$ = a user prompt; $a_i$ = a complete generated response; $R$ = reward model score (helpfulness, safety, etc.)

# Step 2: Group Mean Baseline

Compute the empirical mean reward:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^{K} r_i$$

This is the **group-relative baseline** — the average quality of responses to this prompt.

# Step 2: Group Mean Baseline

Compute the empirical mean reward:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^{K} r_i$$

This is the **group-relative baseline** — the average quality of responses to this prompt.

**Why is this a valid baseline?**

As $K \to \infty$:

$$\bar{r} \xrightarrow{\text{a.s.}} \mathbb{E}_{a \sim \pi_{\text{old}}}[R(q, a)] = V^{\pi_{\text{old}}}(q)$$

With finite $K$, it is a *noisy but unbiased* estimate of the value.

# Step 3: Normalised Advantages

**GRPO advantage estimate**

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$$

where $\sigma_r = \sqrt{\frac{1}{K}\sum_i (r_i - \bar{r})^2}$ and $\epsilon > 0$ for numerical stability.

# Step 3: Normalised Advantages

**GRPO advantage estimate**

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$$

where $\sigma_r = \sqrt{\frac{1}{K}\sum_i (r_i - \bar{r})^2}$ and $\epsilon > 0$ for numerical stability.

- **Centering**: subtracts the baseline (variance reduction)
- **Scaling**: divides by the std. dev. (stabilises gradient magnitudes)
- **Key property**: $\sum_{i=1}^{K} \hat{A}_i = 0$ by construction

# Key Property: Zero-Sum Advantages

$$\sum_{i=1}^{K} \hat{A}_i = \sum_{i=1}^{K} \frac{r_i - \bar{r}}{\sigma_r} = \frac{1}{\sigma_r} \underbrace{\sum_{i=1}^{K} (r_i - \bar{r})}_{=0} = 0$$

**Interpretation**:

- Increasing probability of good responses *must* decrease probability of bad ones
- The update is a **zero-sum redistribution** of probability mass within the group
- This is a form of *contrastive learning* applied to RL

**GRPO turns a reward signal into a relative ranking signal.**

# The GRPO Objective

**GRPO surrogate loss**

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = \frac{1}{K}\sum_{i=1}^{K} \min\Big(\rho_i(\theta)\,\hat{A}_i,\ \mathrm{clip}(\rho_i(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_i\Big)$$

where the **importance ratio** is $\rho_i(\theta) = \dfrac{\pi_\theta(a_i \mid q)}{\pi_{\mathrm{old}}(a_i \mid q)}$.

# The GRPO Objective

**GRPO surrogate loss**

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = \frac{1}{K}\sum_{i=1}^{K} \min\Big(\rho_i(\theta)\,\hat{A}_i,\ \mathrm{clip}(\rho_i(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_i\Big)$$

where the **importance ratio** is $\rho_i(\theta) = \dfrac{\pi_\theta(a_i \mid q)}{\pi_{\mathrm{old}}(a_i \mid q)}$.

- The `clip` prevents large policy updates (PPO-style)

# The GRPO Objective

**GRPO surrogate loss**

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = \frac{1}{K}\sum_{i=1}^{K} \min\Big(\rho_i(\theta)\,\hat{A}_i,\ \mathrm{clip}(\rho_i(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_i\Big)$$

where the **importance ratio** is $\rho_i(\theta) = \dfrac{\pi_\theta(a_i \mid q)}{\pi_{\mathrm{old}}(a_i \mid q)}$.

- The `clip` prevents large policy updates (PPO-style)
- $\hat{A}_i$ is the normalised group advantage from the previous slide

# The GRPO Objective

**GRPO surrogate loss**

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = \frac{1}{K}\sum_{i=1}^{K} \min\Big(\rho_i(\theta)\,\hat{A}_i,\ \mathrm{clip}(\rho_i(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_i\Big)$$

where the **importance ratio** is $\rho_i(\theta) = \dfrac{\pi_\theta(a_i \mid q)}{\pi_{\mathrm{old}}(a_i \mid q)}$.

- The `clip` prevents large policy updates (PPO-style)
- $\hat{A}_i$ is the normalised group advantage from the previous slide
- Optimise over a **mini-batch** of prompts $q$

# Interpreting the GRPO Update

**Responses with $\hat{A}_i > 0$**
(above-average reward)

- Importance ratio $\rho_i > 1$ increases the objective
- Policy is pushed to **make them more likely**

# Interpreting the GRPO Update

**Responses with $\hat{A}_i > 0$**
(above-average reward)

- Importance ratio $\rho_i > 1$ increases the objective
- Policy is pushed to **make them more likely**

**Responses with $\hat{A}_i < 0$**
(below-average reward)

- Importance ratio $\rho_i > 1$ *decreases* the objective
- Policy is pushed to **make them less likely**

# Code: Bandit GRPO — Setup

**Simplest possible GRPO**: a $K$-armed bandit (no state, just actions and rewards).
This is the exact structure of GRPO applied to a single prompt.

```python
import torch
import torch.nn as nn

# Policy over N candidate responses
class BanditPolicy(nn.Module):
    def __init__(self, n_actions=10):
        super().__init__()
        # Learnable logits = log-unnormalised probs
        self.logits = nn.Parameter(
            torch.zeros(n_actions)
        )

    def distribution(self):
        return torch.distributions.Categorical(
            logits=self.logits
        )

def proxy_reward(actions):
    """Stand-in for a reward model.
    Higher action index -> better response."""
    signal = actions.float() * 0.4
    noise  = torch.randn_like(signal) * 0.3
    return signal + noise
```

## Why a bandit?

- No state transitions to worry about
- Each "action" = one generated response
- Reward = score from a reward model
- Captures the *entire* GRPO mechanism

## Mapping to LLMs:

`n_actions` $\approx$ vocabulary$^{\text{seq_len}}$ (exponentially large); in practice we sample rather than enumerate.

# Code: Bandit GRPO — Training Loop

```python
policy    = BanditPolicy(n_actions=10)
optimizer = torch.optim.Adam(
    policy.parameters(), lr=0.05
)

for step in range(500):
    K    = 8  # group size
    dist = policy.distribution()

    # 1. Sample group
    actions = dist.sample((K,))      # [K]
    rewards = proxy_reward(actions)  # [K]

    # 2. GRPO advantages (centred + scaled)
    A_hat = (rewards - rewards.mean()) \
            / (rewards.std() + 1e-8) # [K]

    # 3. Policy gradient (importance ratio = 1
    #    because we re-sample each step)
    log_probs = dist.log_prob(actions)  # [K]
    loss = -(log_probs * A_hat.detach()).mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

**What to watch:**

- `torch.softmax(policy.logits, 0)` — watch probability mass shift toward high-index actions
- `A_hat.sum()` $= 0$ always (by construction)
- Remove normalisation (`std`) and observe gradient instability
- Try $K = 1$ — no update (single sample, $\hat{A}_1 = 0$)

**Exercise**

Replace `proxy_reward` with a function that scores responses by length. Observe reward hacking: policy collapses to the longest action.

Fix: add KL penalty against initial `logits`.

# Why We Need Regularisation

**Without constraints**

Maximising only the reward encourages the policy to drift arbitrarily far from $\pi_{\text{old}}$:

- Importance ratios $\rho_i$ become extreme $\rightarrow$ high-variance updates
- Policy can collapse to degenerate responses (reward hacking)
- Distribution shift invalidates the importance-weighted estimate

# Why We Need Regularisation

**Without constraints**

Maximising only the reward encourages the policy to drift arbitrarily far from $\pi_{\text{old}}$:

- Importance ratios $\rho_i$ become extreme $\rightarrow$ high-variance updates
- Policy can collapse to degenerate responses (reward hacking)
- Distribution shift invalidates the importance-weighted estimate

Solution: add a **KL divergence penalty** to constrain how far the policy moves.

# KL-Regularised Objective

**Regularised GRPO**

$$\max_{\theta} \mathbb{E}_q \left[ \frac{1}{K} \sum_{i=1}^{K} \rho_i(\theta) \hat{A}_i \right] - \beta \, \mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\mathrm{old}}\big)$$

- $\beta > 0$: regularisation strength (hyperparameter)
- $\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{old}}) = \mathbb{E}_\theta \left[ \log \frac{\pi_\theta(a|q)}{\pi_{\mathrm{old}}(a|q)} \right]$
- Larger $\beta$: more conservative updates; smaller $\beta$: more aggressive learning

# Lagrangian Derivation of the Optimal Policy

Constrained form:

$$\max_{\pi} \; \mathbb{E}_q[\mathbb{E}_{a\sim\pi_{\text{old}}}[\rho(\theta)\, A]] \quad \text{s.t.} \quad \mathrm{KL}(\pi\|\pi_{\text{old}}) \leq \delta$$

# Lagrangian Derivation of the Optimal Policy

Constrained form:

$$\max_{\pi} \; \mathbb{E}_{q}[\mathbb{E}_{a\sim\pi_{\text{old}}}[\rho(\theta)\, A]] \quad \text{s.t.} \quad \mathrm{KL}(\pi\|\pi_{\text{old}}) \leq \delta$$

Lagrangian:

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\frac{\pi(a)}{\pi_{\text{old}}(a)}\, A(a)\right] - \lambda\, \mathrm{KL}(\pi\|\pi_{\text{old}})$$

# Lagrangian Derivation of the Optimal Policy

Constrained form:

$$\max_{\pi} \; \mathbb{E}_{q}[\mathbb{E}_{a\sim\pi_{\text{old}}}[\rho(\theta)\, A]] \quad \text{s.t.} \quad \mathrm{KL}(\pi\|\pi_{\text{old}}) \leq \delta$$

Lagrangian:

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\frac{\pi(a)}{\pi_{\text{old}}(a)}\, A(a)\right] - \lambda\, \mathrm{KL}(\pi\|\pi_{\text{old}})$$

Setting $\frac{\delta\mathcal{L}}{\delta\pi(a)} = 0$ and solving:

$$\boxed{\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a,q)}{\lambda}\right)}$$

# Exponential Tilting

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

This is **exponential tilting** (also called Gibbs/Boltzmann distribution):

# Exponential Tilting

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

This is **exponential tilting** (also called Gibbs/Boltzmann distribution):

- Actions with high advantage receive *exponentially* higher probability
- $\lambda$ controls the sharpness (temperature)
- As $\lambda \to 0$: greedy (argmax); as $\lambda \to \infty$: uniform ($= \pi_{\text{old}}$)

# Exponential Tilting

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

This is **exponential tilting** (also called Gibbs/Boltzmann distribution):

- Actions with high advantage receive *exponentially* higher probability
- $\lambda$ controls the sharpness (temperature)
- As $\lambda \to 0$: greedy (argmax); as $\lambda \to \infty$: uniform ($= \pi_{\text{old}}$)

**Information-geometric view**

The KL constraint defines a ball in distribution space.
The optimal policy is the *projection* of the reward-weighted distribution onto this ball.

# GRPO vs. Vanilla Policy Gradient

| | **Policy Gradient** | **GRPO** |
|---|---|---|
| Baseline | Learned critic $\hat{V}_\phi(s)$ | Empirical group mean $\bar{r}$ |
| Extra network | Yes (critic) | **No** |
| Advantage source | TD error | Within-group ranking |
| Bias | Low (w/ good critic) | Finite-$K$ bias |
| Variance | High w/o critic | Low via centering |
| Stability | Critic divergence risk | KL-constrained |

# GRPO for RLHF

# Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

# Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

1. **Ranking-based**: updates depend only on *relative* ordering of responses
   $\Rightarrow$ humans naturally provide rankings, not absolute scores

# Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

1. **Ranking-based**: updates depend only on *relative* ordering of responses
   $\Rightarrow$ humans naturally provide rankings, not absolute scores
2. **Critic-free**: no separate value network to train or stabilise
   $\Rightarrow$ human labels are noisy; a critic would fit noise

# Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

1. **Ranking-based**: updates depend only on *relative* ordering of responses
   $\Rightarrow$ humans naturally provide rankings, not absolute scores
2. **Critic-free**: no separate value network to train or stabilise
   $\Rightarrow$ human labels are noisy; a critic would fit noise
3. **Stable under noisy signals**: normalised advantages $\hat{A}_i = (r_i - \bar{r})/\sigma_r$
   $\Rightarrow$ annotation noise is absorbed by the group standard deviation $\sigma_r$

# Human Feedback as Ranking Data

**What annotators provide:**

- Pairwise comparisons: "A $\succ$ B"
- Rankings of $K$ outputs:
  $a_{\sigma(1)} \succ \cdots \succ a_{\sigma(K)}$
- Preference labels with optional comments

# Human Feedback as Ranking Data

**What annotators provide:**

- Pairwise comparisons: "A $\succ$ B"
- Rankings of $K$ outputs:
  $a_{\sigma(1)} \succ \cdots \succ a_{\sigma(K)}$
- Preference labels with optional comments

**What annotators don't provide:**

- Absolute scores
- Probability values
- Why they prefer one over another

# Human Feedback as Ranking Data

**What annotators provide:**

- Pairwise comparisons: "A $\succ$ B"
- Rankings of $K$ outputs: $a_{\sigma(1)} \succ \cdots \succ a_{\sigma(K)}$
- Preference labels with optional comments

**What annotators don't provide:**

- Absolute scores
- Probability values
- Why they prefer one over another

**Key observation**

Human judgements are **inherently relative**.

"Response A is better than Response B" is easier and more reliable than "Response A scores 7.3 / 10".

**This is exactly what GRPO exploits.**

# Mapping RLHF Concepts onto GRPO

| **RLHF concept** | **GRPO counterpart** |
|---|---|
| Human rankings of $K$ outputs | Group samples $a_1, \ldots, a_K \sim \pi_{\text{old}}$ |
| Preferred response | High reward $r_i \Rightarrow \hat{A}_i > 0$ |
| Dispreferred response | Low reward $r_i \Rightarrow \hat{A}_i < 0$ |
| Implicit quality baseline | Group mean $\bar{r} = \frac{1}{K} \sum r_i$ |
| Reward model score | Scalar $r_i = r_\phi(q, a_i)$ |
| KL constraint (stay safe) | $\beta\, \mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ |

# Key Insight

## The Bridge Statement

**"RLHF is just GRPO where the reward signal comes from humans instead of the environment."**

**Environment reward** (classic RL)

$r_i = R(s, a_i)$ — defined by the MDP
Deterministic or stochastic
Objective: maximise $J(\theta)$

**Human reward** (RLHF)

$r_i = r_\phi(q, a_i)$ — predicted by reward model
Noisy, biased, context-dependent
Objective: align with human intent

# Why This Matters

The GRPO/RLHF correspondence has three important consequences:

# Why This Matters

The GRPO/RLHF correspondence has three important consequences:

1. **No true reward function required**
   We replace the intractable "true reward" with human comparative judgements

# Why This Matters

The GRPO/RLHF correspondence has three important consequences:

1. **No true reward function required**
   We replace the intractable "true reward" with human comparative judgements
2. **Reduced dependence on value function estimation**
   The group mean $\bar{r}$ is a computable proxy — no learned critic needed
   This is critical when the state space is natural language

# Why This Matters

The GRPO/RLHF correspondence has three important consequences:

1. **No true reward function required**
   We replace the intractable "true reward" with human comparative judgements
2. **Reduced dependence on value function estimation**
   The group mean $\bar{r}$ is a computable proxy — no learned critic needed
   This is critical when the state space is natural language
3. **Relative structure arises naturally**
   Human preference is ordinal; GRPO training signal is ordinal
   No mismatch between data structure and algorithm

# Practical Application:
# Training LLMs with Verifiable Rewards

- **No Reward Model (RM) needed**: For tasks with a ground truth, we use rule-based code to verify answers.
- **Massive Throughput**: We generate multiple answers in parallel to build our reference group baseline.

# DeepSeek-R1 Insight: Verifiable Rewards

Why does this work so well for Math and Code? We replace noisy human-like scoring with **objective rules**.

### Accuracy Reward

Does the answer match the target?

- Correct: $+1.0$
- Incorrect: 0.0

### Format Reward

Does it follow the structure?

- Used `<thought>`?
- Used `\boxed{}`?

*This removes "Reward Model bias" because the signal is factual truth.*

# A Concrete Example: The "Tournament"

**Prompt**: "What is the square root of 144? Answer in a box."

| ID | Model Content | Correct? | Reward | Advantage |
|---|---|---|---|---|
| $a_1$ | "Answer is $\boxed{12}$." | Yes | 1.0 | $+0.98$ |
| $a_2$ | "$12 \times 12 = 144$. $\boxed{12}$" | Yes | 1.0 | $+0.98$ |
| $a_3$ | "It is 14. $\boxed{14}$" | No | 0.0 | $-1.02$ |
| $a_4$ | "I think it is $\boxed{72}$" | No | 0.0 | $-1.02$ |

- **Group Baseline**: Mean $\bar{r} = 0.5$.
- **Learning Signal**: Responses $a_1$ and $a_2$ are reinforced. $a_3$ and $a_4$ are suppressed.
- **Outcome**: The model discovers that valid math and formatting lead to "winning."

# Lecture 3 — Summary

**Core take-aways**

1. **GRPO** replaces the expensive critic with a group-mean baseline.
2. **Verifiable Rewards** (Math/Code) make GRPO extremely stable.
3. Centering rewards ($\hat{A}_i = \frac{r_i - \bar{r}}{\sigma}$) provides the variance reduction.
4. **KL regularisation** is critical to prevent the model from "cheating."
5. **DeepSeek-R1** proved that this is the path to competitive reasoning.
6. RLHF = GRPO logic, but with human rankings as the reward source.

Next: When does this work in theory? (Lecture 4 — Convergence)